

ICS Assignment 1

Venkata Sreekar

March 2026

Part 1

1 Exercise 1

1.2 Theoretical Comparison

- **DFF:** always replaces stored value with new input (it is like load is always equal to 1).
- **1-Bit Register:** replaces the value only when asked with extra input load.

1.3 Implementation

We do not need register for this, we can implement the counter using DFF in the following way.

- We take the output from the DFF and pass it through a Not gate.
- Then we pass it as the input for the DFF.

every time the clock ticks, DFF takes the input (which is Not of previous output), so the output after each tick will Not of previous output.

Whereas the counter will also be Not of it's previous value, the first value of the counter is Not of first value of output of DFF.

So, at every instant, the counter is Not of output of DFF.

3 Exercise 3

1. **NO**, we cannot implement 2 8-bit register using a 16-bit register, because to implement 2 8-bit register, we need 2 loads (1 for each of the 8-bit register), whereas in a 16-bit register, we only have 1 load.
2. we need an extra **input for load** of other register. So total we need 2 inputs for the load.
3.
 - **Advantages of 8-Bit register:** can update a part of word without completely updating the whole word,
 - **advantages of 16-Bit register:** can update whole word in 1 clock, No need for byte selection, Simpler control logic.

4 Exercise 4

1. RAM8 contains 8 registers, each storing 16 bits.
To read or write a specific register, we use an address input.

If load = 1, we update one of the 8 registers (decided by the address)

If load = 0, we read one of the 8 registers (decided by the address)

This part is done in the same way as above built 16-bit register (using DFF and Mux).

2.
 - **INPUT :**
in[0:15] → data to store,
load → write enable,
address[0:2] → selects which register to read/write.
 - **OUTPUT :**
out[0:15] → (data stored in the selected register)
3. As there are 8 registers, we need 3 binary inputs to uniquely define one of those 8 registers

$$8 = 2^3$$

So, we need 3 address bits Address[0], Address[1], Address[2].

5 Exercise 5

1. $1 \text{ kilobit} = 2^{10} \text{ bits}$
 $1 \text{ kilobit} = 1024 \text{ bits}$

 $1 \text{ Kb is equal to } 1024 \text{ bits.}$
2. $1 \text{ kilobit} = 2^{10} \text{ bytes}$
 $1 \text{ kilobit} = 1024 \text{ bytes}$
 $1 \text{ kilobit} = 8 \times 1024 \text{ bits}$

 $1 \text{ KB is equal to } 1024 \text{ bytes,}$
 $1 \text{ KB is equal to } 8192 \text{ bits.}$
3. $1 \text{ kilobit} = 8192 \text{ bits}$
 $16 \text{ kilobit} = 16 \times 8192 \text{ bits}$

 $16 \text{ KB is equal to } 131072 \text{ bits.}$

6 Exercise 6

To build **1 KB** memory, we need 1024 words of 16-bits

1. **RAM8 Chips**
each RAM8 chip stores 8 words of 16-bit each,
So we need
$$\frac{1,024}{8} = 128 \text{ chips}$$
2. **RAM16 Chips**
each RAM16 chip stores 16 words of 16-bit each,
So we need
$$\frac{1,024}{16} = 64 \text{ chips}$$

3. RAM64 Chips

each RAM64 chip stores 64 words of 16-bit each,
So we need

$$\frac{1,024}{64} = 16 \text{ chips}$$

7 Exercise 7

1. • RAM8:

as it stores 8 words, we need 3 addresses inside RAM8.

also we need 128 chips for 1KB, so we need 7 addresses to select of one RAM8 chip.

So, total we need $3 + 7 = 10$ address lines.

• RAM64:

as it stores 64 words, we need 6 addresses inside RAM64.

also we need 16 chips for 1KB, so we need 4 addresses to select of one RAM64 chip.

So, total we need $6 + 4 = 10$ address lines.

2. No, it does not change the total number of address bits required by the system. The total number of address bits is determined by the total number of memory registers you want to access, entirely independent of how you physically build the chip.

If you build with larger block, you will require less number of chips, but you will require more Address bits to build the larger block.

$$2^{\text{\#Address Bits}} = \text{Total Memory Registers}$$

3. I first built a RAM8 block using 16-bit register, using that I built RAM64 using RAM8, RAM512 using RAM64, .. upto RAM16k using RAM4k. (All those files are submitted in the .zip file).

8 Extra Credit

1. • **RAM8:** Building a large memory module from many small blocks like RAM8 creates a complex routing bottleneck that significantly impacts performance for high-speed AI tasks.
• **RAM4k:** In contrast, utilizing larger contiguous blocks such as RAM4K optimizes the hardware for the high-bandwidth data streaming essential to modern neural networks.
2. • **RAM8:** From a manufacturing and cost standpoint, implementing a 16KB memory using 1,000+ RAM8 chips is more expensive and architecturally inefficient. The design requires an immense number of external pins and a massive inter-chip routing network, which drastically increases the physical size and complexity.
• **RAM1k:** Whereas, using eight RAM1K chips is cost-effective approach for modern system design. By aggregating memory into fewer, higher-density blocks, you reduce the address decoding paths significantly

Part 2

1 Exercise 1

1. This part is explained in the 1st part of 4th question of this exercise.
2. As we have already assigned the 18 encoding to the standard instructions, we should use a encoding which does the effect the already existing instructions.

I used 2 encoding, '101001', '111000'. One for left 16-bit part (msb) and other for the right 16-bit part (lsb).

I showed that these encodings are valid in 3rd question of this exercise.

3.
 - I have added `multiplier.tst` and `multiplier.cmp` files in the submission in the .zip file.
 - I used 2 encodings ('101001', '111000'), neither of them are in the 18 default instructions in the ALU file.

In the ALU file I have submitted, It calculates all 3 values (standard output for the encoding, leftMUL, rightMUL).

And it decides which one to output based on the given encoding using MUX.

```
Mux16(a=standardOut, b=mulLow, sel=isMulLow, out=tempOut);  
Mux16(a=tempOut, b=mulHigh, sel=isMulHigh, out=out, ...);
```

if both isMulHigh and isMulLow are zero (the encoding is neither '101001' nor '111000'), from the 1st Mux,

`tempOut = standardOut.` (As `isMulLow = 0`)

and, for the 2nd Mux,

`out = tempOut = standardOut.` (As `isMulHigh = 0`)

So, if the encoding is neither '101001', '111000' it will just give the standardOut. (Which does not impact the already existing ALU instructions.)

4.
 - A Wallace tree multiplier speeds up multiplication by avoiding, step-by-step addition of partial product rows. Instead, it compresses bits in parallel columns, reducing the computation time from about $O(N)$ to $O(\log N)$ **Nearly Constant**. It mainly uses Full Adders (3:2 compressors) to combine three bits into two, and Half Adders to handle remaining pairs. This process continues until we remain with 2 numbers. These final two rows are added with a Carry Propagate Adder reduces the total time greatly.
 - I used 2 encodings for the multiplication (101001, 111000), one for the left part of the 32-bit output and one for the right part of the 32-bit output.
`zx = 1, nx = 0, zy = 1, ny = 0, f = 0, no = 1` (for left half)
`zx = 1, nx = 1, zy = 1, ny = 0, f = 0, no = 0` (for right half)
 - If one of the input for the multiplication is of the form 2^N , then it is equivalent to do bit shift by N bits to the left.

(OR)

If one of the input is of the form $2^N + 1$, then,

$$X \times (2^N + 1) = X \times 2^N + X$$

we first calculate $X \times 2^N$ (which is N bit shift to left side) and add it to X (by carry propagate adder).

Extra Credit Point

Recent favorite song : Niluvaddham from Nuvvostanante Nenoddantana movie.